

Unit 3

Unit 3: Angular, React, TypeScript — Practical/Oral Exam Notes

Context

Part of [Web Application Development](#) | Practical + Oral Exam Prep | SPPU Third Year IT

High-Yield Oral Topics

- **Angular Architecture** — Modules, Components, Services, DI
- **Data Binding** — 4 types, especially two-way `[(ngModel)]`
- **Directives** — `*ngIf` vs `*ngFor` (structural vs attribute)
- **React Hooks** — `useState`, `useEffect`, `useContext`
- **Angular vs React** — Always compared in viva
- **MVC Architecture** — How it maps to frameworks

MINI PROJECT ANCHOR: Student Management System (SMS)

Angular App: Register Student → Login → View Profile/Dashboard

SECTION 1 — MVC ARCHITECTURE

Exam Definition

MVC (Model-View-Controller) is a **software design pattern** that separates an application into three interconnected components to organize code and separate concerns.

Layer	Role	SMS Example
Model	Data + Business Logic	Student schema, DB queries
View	UI shown to user	HTML templates, Angular components
Controller	Bridge — processes requests	Express route handlers

MVC Flow (Viva Answer)

"User submits a registration form (**View**). The form data goes to the **Controller** which validates it and calls the **Model** to save to MongoDB. The Model returns success. The Controller sends a response. The View updates to show the new student."

MVC in Angular

- **Model** = TypeScript interfaces/classes (`student.model.ts`)
 - **View** = Component template (`.component.html`)
 - **Controller** = Component class (`.component.ts`)
-

SECTION 2 — TYPESCRIPT

📌 Exam Definition

TypeScript is a **statically typed superset of JavaScript** developed by Microsoft. It adds type safety and OOP features, then **compiles to plain JavaScript**.

Key Features

```
// 1. Static Typing – catch errors at compile time
let studentName: string = "Rahul";
let rollNo: number = 101;
let isActive: boolean = true;

// 2. Interface – defines shape of an object
interface Student {
  id: number;
  name: string;
  course: string;
  marks?: number[]; // ? = optional field
}

// 3. Typed function
function getAverage(marks: number[]): number {
  return marks.reduce((sum, m) => sum + m, 0) / marks.length;
}

// 4. Class with types
class StudentService {
  private students: Student[] = [];

  addStudent(student: Student): void {
    this.students.push(student);
  }

  getStudents(): Student[] {
    return this.students;
  }
}
```

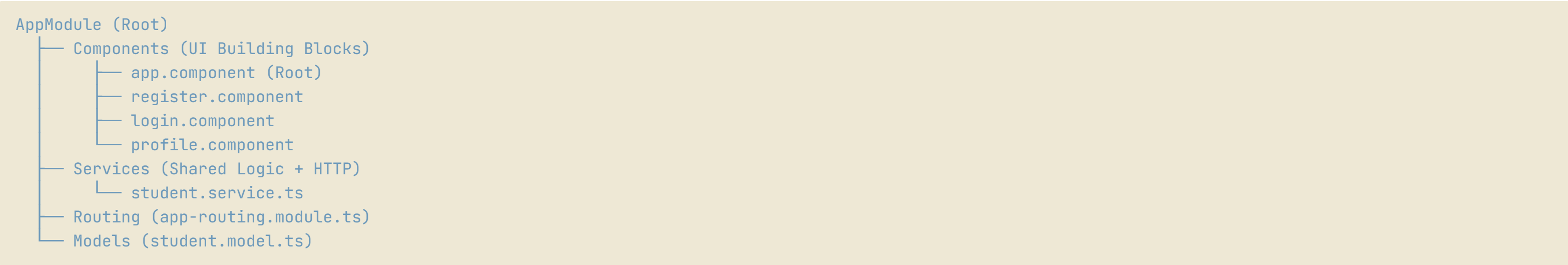
```
}  
}  
  
// 5. Modules - import/export  
export interface Student { ... }  
import { Student } from './student.model';
```

JS vs TypeScript

Feature	JavaScript	TypeScript
Typing	Dynamic (runtime errors)	Static (compile-time errors)
OOP	Partial	Full (interfaces, generics)
IDE Support	Basic	Excellent autocomplete
Compilation	Runs directly	Compiles to JS first
Error Detection	Runtime	Compile time

SECTION 3 — ANGULAR

Angular Architecture Diagram



Angular CLI Commands — Must Know

```
# Install Angular CLI globally  
npm install -g @angular/cli  
  
# Create new project  
ng new student-management-system  
cd student-management-system
```

```
# Generate components
ng generate component register
ng generate component login
ng generate component profile

# Generate service
ng generate service student

# Run development server
ng serve           # Runs on http://localhost:4200

# Build for production
ng build --prod
```

SMS Angular — Complete Register Component

```
// register.component.ts
import { Component } from '@angular/core';
import { StudentService } from '../student.service';
import { Router } from '@angular/router';

@Component({
  selector: 'app-register',
  templateUrl: './register.component.html',
  styleUrls: ['./register.component.css']
})
export class RegisterComponent {
  // Two-way binding properties
  student = {
    name: '',
    email: '',
    course: '',
    password: ''
  };

  successMessage = '';
  errorMessage = '';

  // Dependency Injection – Angular provides StudentService automatically
  constructor(
    private studentService: StudentService,
    private router: Router
```

```

    ) {}

    // Called on form submit
    onSubmit() {
      this.studentService.registerStudent(this.student).subscribe({
        next: (response) => {
          this.successMessage = 'Registration successful!';
          setTimeout(() => this.router.navigate(['/login']), 1500);
        },
        error: (err) => {
          this.errorMessage = err.error.message || 'Registration failed';
        }
      });
    }
  }
}

```

```

<!-- register.component.html -->
<div class="container mt-5">
  <div class="row justify-content-center">
    <div class="col-md-6">
      <div class="card">
        <div class="card-header bg-primary text-white">
          <h4>Student Registration</h4>
        </div>
        <div class="card-body">

          <!-- Success/Error Messages -->
          <div *ngIf="successMessage" class="alert alert-success">
            {{ successMessage }}
          </div>
          <div *ngIf="errorMessage" class="alert alert-danger">
            {{ errorMessage }}
          </div>

          <!-- Form with Two-Way Binding -->
          <form (ngSubmit)="onSubmit()" #regForm="ngForm">

            <div class="mb-3">
              <label>Full Name</label>
              <!-- [(ngModel)] = Two-way data binding -->
              <input type="text" class="form-control"
                [(ngModel)]="student.name" name="name" required>
            </div>

```

```

<div class="mb-3">
  <label>Email</label>
  <input type="email" class="form-control"
    [(ngModel)]="student.email" name="email" required>
</div>

<div class="mb-3">
  <label>Course</label>
  <select class="form-select" [(ngModel)]="student.course" name="course">
    <option value="IT">Information Technology</option>
    <option value="CS">Computer Science</option>
  </select>
</div>

<div class="mb-3">
  <label>Password</label>
  <input type="password" class="form-control"
    [(ngModel)]="student.password" name="password" required>
</div>

<button type="submit" class="btn btn-primary w-100"
  [disabled]="regForm.invalid">Register</button>
</form>

<p class="mt-3 text-center">
  Already registered? <a routerLink="/login">Login here</a>
</p>
</div>
</div>
</div>
</div>
</div>

```

Student Service — HTTP Calls

```

// student.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root' // Available throughout the app (singleton)

```



```

})

export class StudentService {
  private apiUrl = 'http://localhost:3000/api';

  // HttpClient injected via DI
  constructor(private http: HttpClient) {}

  registerStudent(data: any): Observable<any> {
    return this.http.post(`${this.apiUrl}/students`, data);
  }

  loginStudent(credentials: any): Observable<any> {
    return this.http.post(`${this.apiUrl}/login`, credentials);
  }

  getAllStudents(): Observable<any[]> {
    return this.http.get<any[]>(`${this.apiUrl}/students`);
  }

  deleteStudent(id: string): Observable<any> {
    return this.http.delete(`${this.apiUrl}/students/${id}`);
  }
}

```

Angular Routing Setup

```

// app-routing.module.ts
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { RegisterComponent } from './register/register.component';
import { LoginComponent } from './login/login.component';
import { ProfileComponent } from './profile/profile.component';

const routes: Routes = [
  { path: '', redirectTo: '/login', pathMatch: 'full' },
  { path: 'register', component: RegisterComponent },
  { path: 'login', component: LoginComponent },
  { path: 'profile', component: ProfileComponent },
  { path: '**', redirectTo: '/login' } // Wildcard - 404 redirect
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],

```

```
    exports: [RouterModule]
  })
  export class AppRoutingModule {}
}

<!-- app.component.html - Router Outlet -->
<nav class="navbar navbar-dark bg-dark">
  <a routerLink="/register" routerLinkActive="active">Register</a>
  <a routerLink="/login" routerLinkActive="active">Login</a>
  <a routerLink="/profile" routerLinkActive="active">Profile</a>
</nav>

<!-- Component renders here based on route -->
<router-outlet></router-outlet>
```

Data Binding — 4 Types (MUST KNOW)

Type	Syntax	Direction	Example
Interpolation	{{ value }}	Component → View	{{ student.name }}
Property Binding	[property]="value"	Component → View	[disabled]="isLoading"
Event Binding	(event)="method()"	View → Component	(click)="onSubmit()"
Two-Way Binding	[(ngModel)]="var"	Both ways	[(ngModel)]="student.name"

Directives Quick Reference

```
<!-- *ngIf - Structural: add/remove from DOM -->
<div *ngIf="isLoggedIn">Welcome, {{ user.name }}!</div>
<div *ngIf="!isLoggedIn">Please login.</div>

<!-- *ngFor - Structural: repeat element -->
<tr *ngFor="let s of students; let i = index">
  <td>{{ i + 1 }}</td>
  <td>{{ s.name }}</td>
  <td>{{ s.course }}</td>
</tr>

<!-- ngClass - Attribute: dynamic CSS -->
<div [ngClass]="{ 'active': isActive, 'error': hasError }"></div>

<!-- ngStyle - Attribute: dynamic inline style -->
<p [ngStyle]="{ 'color': student.marks > 75 ? 'green' : 'red' }">
```



```
    {{ student.marks }}
  </p>
```

Lifecycle Hooks

Hook	When it fires	Use Case
ngOnInit	After component created	API calls, initialization
ngOnChanges	When @Input() value changes	React to parent data
ngOnDestroy	Before component removed	Cleanup subscriptions

```
export class ProfileComponent implements OnInit, OnDestroy {
  students: any[] = [];
  private subscription: any;

  ngOnInit() {
    // Load data when component initializes
    this.subscription = this.studentService.getAllStudents()
      .subscribe(data => this.students = data);
  }

  ngOnDestroy() {
    // Prevent memory leak – unsubscribe
    this.subscription.unsubscribe();
  }
}
```

SECTION 4 — REACT (OVERVIEW)

React Hooks — SMS Example

```
import React, { useState, useEffect } from 'react';

function StudentList() {
  // useState – local state management
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // useEffect – side effects (API calls, DOM updates)
```

```

useEffect(() => {
  // Runs after component mounts (like ngOnInit)
  fetch('http://localhost:3000/api/students')
    .then(res => res.json())
    .then(data => {
      setStudents(data);
      setLoading(false);
    })
    .catch(err => {
      setError(err.message);
      setLoading(false);
    });

  // Cleanup function (like ngOnDestroy)
  return () => {
    console.log('Component unmounted');
  };
}, []); // [] = run once on mount

if (loading) return <div>Loading...</div>;
if (error) return <div>Error: {error}</div>;

return (
  <table className="table">
    <thead>
      <tr><th>Name</th><th>Course</th></tr>
    </thead>
    <tbody>
      {students.map(s => (
        <tr key={s._id}>
          <td>{s.name}</td>
          <td>{s.course}</td>
        </tr>
      ))}
    </tbody>
  </table>
);
}

```

Feature	Angular	React
Type	Full Framework	UI Library
Language	TypeScript	JavaScript/JSX
DOM	Regular DOM	Virtual DOM
Data Binding	2-way by default	1-way by default
State Management	Services + RxJS	useState + Redux
Learning Curve	Steep	Moderate
Opinionated	Yes (strict structure)	No (flexible)
Backed by	Google	Meta (Facebook)
Best For	Enterprise apps	Dynamic SPAs

VIVA Q&A — Unit 3

Q1: What is Dependency Injection in Angular?

DI is a design pattern where Angular automatically provides instances of dependencies (like services) to components that need them. Instead of creating `new StudentService()` manually, you declare it in the constructor and Angular injects it. Benefits: Loose coupling, easier testing, single instance (singleton) across app.

Q2: What is `*ngFor` vs `*ngIf` ?

Both are **structural directives** that modify DOM structure. `*ngIf` conditionally shows/removes an element. `*ngFor` repeats an element for each item in an array. They both use the `*` prefix which means they can add/remove DOM elements.

Q3: What is a Service in Angular?

A Service is a class with `@Injectable()` decorator that contains **shared business logic** (like HTTP calls). Services are **singleton** — one instance is shared across all components. They're injected via DI. Example: `StudentService` with `register()`, `login()`, `getAll()` methods used by multiple components.

Q4: What is the difference between `Observable` and `Promise` ?

Promise handles a **single** async event. Observable handles **streams** of events over time and can be cancelled. Angular's `HttpClient` returns Observables. You subscribe with `.subscribe()`. Observables support operators like `map`, `filter` from RxJS.

Q5: Explain `useState` and `useEffect` in React.

`useState(initialValue)` gives a component memory — it returns `[value, setter]`. Changing value via setter re-renders the component. `useEffect(fn, deps)` runs side effects after render — for API calls, DOM updates, subscriptions. The `deps` array controls when it runs: `[]` = once on mount, `[var]` = when var changes.

Oral Trap Questions

Trap: "What is `routerLink` vs `href` in Angular?"

`href` causes a full page reload — it's for regular links. `routerLink` is Angular's directive that navigates WITHOUT reloading the page. It uses the Angular Router to load the component dynamically. Always use `routerLink` inside Angular apps.

Trap: "Can you use `*ngIf` and `*ngFor` on the same element?"

No! Angular doesn't allow two structural directives on the same element. Solution: wrap in an `<ng-container>` which renders no extra DOM element.

PRACTICAL FILE — Assignment 2 (Angular Part)

Viva explanation:

"I created an Angular application with 3 components: Register, Login, and Profile. The Register component uses two-way data binding `[(ngModel)]` to bind form fields to a TypeScript object. On submit, it calls `StudentService.registerStudent()` which uses `HttpClient.post()` to call our Node.js REST API. The Router navigates to Login after success. The Profile component loads all students using `ngOnInit` and displays them with `*ngFor`."

High Probability Questions

1. Explain Angular Architecture with diagram
2. What are the 4 types of data binding in Angular?
3. Difference between structural and attribute directives
4. What is Dependency Injection? Why use it?
5. Explain `ngOnInit` lifecycle hook
6. What is `[(ngModel)]`? How does two-way binding work?
7. Angular vs React comparison
8. What is `useState` in React?
9. What is a Service in Angular?
10. What is routing in Angular? How to configure it?